column-hse-ltau: implementation

Technical notes on `run.mode = "column-hse-ltau"`. HSEMHS, 26 Oct 2026 (after the embedding-API merge, 9d89191).

1. Running it

```
[mpiexec -n R] julia --project=. -t T scripts/run.jl config/modes/column-hse-ltau.toml
```

Driver `run_column_hse_ltau` (`src/RunModes.jl`): read the file, `prepare_embedded(COMM_WORLD, :column_hse_ltau, ...)`, `solve_embedded!`, write the file, `free_embedded!`. Physics in `src/PoissonTau.jl` (`hse_tau!`, `align_ptot!`, `gauge!`, `column_hse_tau`); the in-memory API in `src/Embedded.jl`; MPI layer in `src/Slab.jl`. The same computation is available without files through the embedded API (Sect. 5).

Key	Default	Meaning
`input.file`	required	τ-grid cube (layout below)
`input.x`, `input.y`	`x`, `y`	names of the coordinate datasets
`physics.g`	`"from_input"`	number, or the file attribute `gravity_m_s2` (error if absent)
`eos.table_file`	`cache/eos_table_fullz.h5`	EOS and opacity table
`tau_grid.align`	`ptot`	`ptot`: equal $P + B^2/2\mu_0$ at one height; `none`: $z(\tau{=}1) = 0$ per column
`run.output`, `run.overwrite`	required, false	output file

The key `tau_grid.cap_per_dex` no longer exists (removed in c8bf91e); a configuration that still sets it is rejected as an unknown key.

2. Input layout and checks

`IO/input-mhs-ltau.nc`: `log10tau500` (n_τ, increasing: top level first), `x` (N_x), `y` (N_y), `temperature`, `B_x`, `B_y`, `B_z` with h5py shape (x, y, τ), i.e. Julia (τ, y, x). The driver first reads the coordinates (`tau_coords`) on every rank, then each rank reads only its x hyperslab (`read_tau_input`). The reader also accepts the `make_tau_input.py` test layout (`logtau500`, `x_m`, `y_m`, `T`, `Bx...`, h5py (τ, y, x)).

- **Units:** a `units` attribute on `x`, `y` is converted (m, km, cm, Mm); `temperature` and the field are read *as is*, assumed to be K and T. A field stored in G is not converted.
- **Names:** only the coordinate names are configurable (`input.x`, `input.y`); `temperature`, `B_x`, `B_y`, `B_z` and `log10tau500` are fixed. The other `[input]` keys are not used.
- **Checks** (`prepare_embedded`, agreed on all ranks before any collective step): at least 3 levels, $\log\tau$ finite and *strictly* increasing; x, y finite, strictly increasing and uniform to 10^{-9} relative; g finite and positive; `align` valid; at least 2 x columns per rank ($\lfloor N_x/R \rfloor \geq 2$). In `solve_embedded!`: slab shapes and finiteness of $\ln T$ and $\mathbf{B}$ ($\mathbf{B}$ required for `align = "ptot"`). Any failure is an `InvalidInput` raised identically on every rank.

3. Algorithm

1. `hse_tau!` per column, top down. Top: fixed point $\ln P \leftarrow \frac{1}{2}\ln P + \frac{1}{2}\ln(g\tau_1\rho/\chi)$ from $P = 100\,\mathrm{Pa}$ until the change is below 10^{-10} (at most 200 iterations); a non-finite result fails the column.
2. Between levels $a = \ln\tau_k$, $b = \ln\tau_{k+1}$: Heun (explicit trapezoid) substeps of $(\ln P, z)$ with $f = g\tau\rho/(\chi P)$, $g_z = -\tau/\chi$, $\ln T$ linear in $\ln\tau$. Substep: $h = \min(b - x,\ 0.05/|f_1|)$; evaluate f_2 at $x + h$ with the Euler predictor; while $h|f_2| > 0.1$ (and $h > 10^{-9}(b - a)$), $h \leftarrow h/4$ and re-evaluate. Then $\ln P \mathrel{+}= \frac{h}{2}(f_1 + f_2)$, $z \mathrel{+}= \frac{h}{2}(g_{z1} + g_{z2})$. The end-slope condition (b73c863) resolves hot-over-cool levels: with the start slope alone, one step covered a whole 0.1-dex level across which the slope grows $\sim 10^3$-fold, giving $\Delta\ln P \approx 21$ ($P \times 10^9$) in some columns of a 504^2 MURaM run.
3. **No cap on the rise of** $\ln P$ (c8bf91e). The former `cap_per_dex` limit (50 per dex) also stopped the substep loop without advancing z, which left adjacent levels at the same height (1683 columns of `IO/input-mhs-ltau.nc`). Since $P = gm$ and $dz = -dm/\rho$, heights now strictly decrease.
4. **Failure:** if f_1 or f_2 is non-finite or the new $\ln P$ exceeds `tab.Y[end]`-1, the column's $\ln P$ from that level down and all its z are set to NaN and `hse_tau!` returns `false`. `column_hse_tau` counts failed columns and non-finite levels globally; if there are any it skips the alignment and gauge (they would spread the NaNs), the status becomes `:eos_out_of_envelope`, and the file driver raises an error (non-zero exit, no output).
5. z is shifted so that $z(\log\tau = 0) = 0$ (linear in $\log\tau$, `interp_lt`, extrapolated from the end segments).
6. `align_ptot!`: reference $\ln(P + B^2/2\mu_0)$ = global median over the bottom level (exact distributed median); per column the height where the total pressure crosses it, going up from the bottom (linear in z between levels in $\ln P_t$), or, below the bottom, $z_{\rm bot} - \Delta\ln P_t\,/(g\rho/P)_{\rm bot}$: the *gas* pressure scale height applied to the total pressure,

an approximation. The column is shifted by that height. The tension is ignored.
7. `gauge!`: subtract the global mean of $z(\log\tau = 0)$.
8. `tau_state!`: ρ, n_e, χ_{500} and the EOS flags from one table call per node; $z(\tau{=}1)$.
9. `check_fields!`: EOS flags clean, all fields finite, $P, \rho, \chi > 0$, z strictly decreasing and τ strictly increasing down every column. The first failed check becomes the status reason (`:nonfinite`, `:nonpositive`, `:folded`). Note: the file driver errors only on `:eos_out_of_envelope`; a result failing a later check is still written and the run exits 0 (code follow-up). On `IO/input-mhs-ltau.nc`: 0 failed, 0 folded columns.

The field values are carried unchanged to the new heights; nothing enforces $\nabla \cdot \mathbf{B} = 0$ in physical space.

4. MPI layer (`Slab.jl`, `Parallel.jl`)

Shared with the two Poisson modes.

- `slab_par(comm, NX, NY, n)` → `init_parallel(comm, ...; px = R, py = 1)`: each rank owns a contiguous range of x columns (`block_range`) and all of y. The parent communicator is *duplicated* (`Comm_dup`) and the Cartesian communicator built from the duplicate, so HSEMHS messages cannot match the caller's; both are owned by the `ParCtx` and released by `free_parallel!`. The standalone `slab_par(NX, NY, n)` initialises MPI if needed and uses `COMM_WORLD`. Arrays are (k, j, i) = (vertical, y, x), columns contiguous.
- Reductions: `gsum`, `gdot`, `gnorm`, `gmax`, `gcount` (Allreduce).
- `gquantiles`/`gmedians`: *exact* global order statistics without gathering. Each Float64 is mapped to an order-preserving Int64 key; per set the k-th smallest key is found by bisection on the key range, one Allreduce of the per-set counts per round (at most 64 rounds), all sets at once. Type-7 quantiles as `Statistics.quantile`.
- `coop_write`: rank 0 creates every dataset at its global size (Julia $(x, \dots)$, h5py $(\dots, x)$) and the attributes, then the ranks write their x hyperslabs in turn between barriers (no parallel HDF5).
- The column integration has no communication; the alignment and gauge use global reductions (median exact, mean to rounding), so results agree across rank counts to rounding (measured $\leq 3 \times 10^{-12}$ relative, Sect. 7).

Threads: owned columns in a threaded loop over x. `scripts/run.jl` sets BLAS to one thread (HSEMHS itself no longer touches BLAS or FFTW threads on load).

5. Embedded API

For callers that own MPI (e.g. an inversion code): `ws = prepare_embedded(comm, :column_hse_ltau; axis = log10tau, x, y, eos, g, column = ColumnTauOptions(align = "ptot"))`, then `res = solve_embedded!(ws, lnT, Bx, By, Bz)` per evaluation and finally `free_embedded!(ws)`; all three are collective. Arrays are the rank's x slab in (vertical, y, x) order; rank r owns `slab_range(NX, R, r)`. The result (`EquilibriumResult`) holds `lnP`, `Pgas`, `rho`, `ne`, `chi500`, `tau500`, `logtau500`, `z` (aligned, gauged), `z_tau1`, `eos_flags`, in workspace buffers overwritten by the next solve (`snapshot` copies). The status (`EquilibriumStatus`, identical on all ranks) carries `valid`, `reason`, `counts`, `messages` and `stats` (`columns_failed`, `align_total_pressure_Pa`, `align_extrapolated_columns`). Nothing in the API initialises MPI, uses `COMM_WORLD`, sets threads, prints or touches files (except loading a table given as a path).

6. Output

`/tau_grid/{logtau500, z_m, Pgas, rho, T, Bx, By, Bz, z_tau1_m}` and `/coords/{x_m, y_m}`; arrays Julia (x, y, τ), h5py (τ, y, x). Attributes: `mode`, `input`, `eos_table`, `g`, `align`, `mpi_ranks`, `threads`, `units` (`cap_per_dex` and `capped_intervals` are gone).

7. Verification and performance

- Tests (in `test/runtests.jl`): `test_embedded.jl` compares the embedded result with stored standalone outputs on two windows of the production input (24×18, 11×12 columns) to $\leq 10^{-12}$ and checks layout, option and input validation, the out-of-envelope status and the field checker; `mpi/mpi_embedded.jl` on 2, 3, 4 ranks: relative differences to one rank $\leq 3 \times 10^{-12}$ (asserted 10^{-11}), plus collective-failure cases.
- `test_tau_height.jl`: an 8400 K over 3000 K level integrated on the 0.1-dex grid matches the same column on a 10× finer grid to 0.05 in $\ln P$, with photospheric pressure ($< 10^5$ Pa) at the bottom.
- The rewiring through the embedded API reproduced the pre-refactor driver outputs bitwise; the cap removal and the end-slope rule then changed the columns that had been capped or under-resolved (the reference outputs were regenerated in b73c863).
- On the 504^2 MURaM run with the end-slope rule (from the commit record): max P $9.7 \times 10^8 \to 9.4 \times 10^4$ Pa, 0 columns above 10^6 Pa, 0 failed columns; $\ln P$ agrees with a 10× tighter step limit to 4×10^{-3} (median) and 0.05 (max) per column.
- Full production cube timing (24 Oct, 13666b7, still with the cap): 8 ranks × 32 threads on 4 nodes, 95 s wall, mostly start-up and I/O. The current code has not been timed on the full cube; the end-slope rule adds substeps

only in the steep intervals.